NEXT.JS 16 & REACT 19 NON-PROFIT CAMPUS PLATFORM THREE.JS WEBGL

CAMPUS-SHARE

Peer-to-Peer & Lab-to-Peer Hardware Lending Platform and Open-Source 3D CAD Repository for University Engineering Communities

Document Title: System Architecture & API Documentation

Project Version: 1.0.0 (Production Release)

Tech Stack: Next.js 16, TypeScript, Tailwind CSS v4, Prisma, Three.js

Security: Universal Academic Email Auth, NextAuth JWT, RBAC

Date Generated: October 2026

Target Environment: University Engineering Departments & Makerspaces

§ 1. Executive Overview & Mission

Campus-Share is an open-source, non-profit academic resource platform engineered to bridge resource inequities across university engineering faculties. It allows students, project groups, and faculty labs to lend and borrow microcontrollers, sensors, actuators, and precision test equipment without monetary deposits, while sharing validated 3D CAD files (.STL, .STEP, .OBJ) with real-time in-browser WebGL mesh inspection.

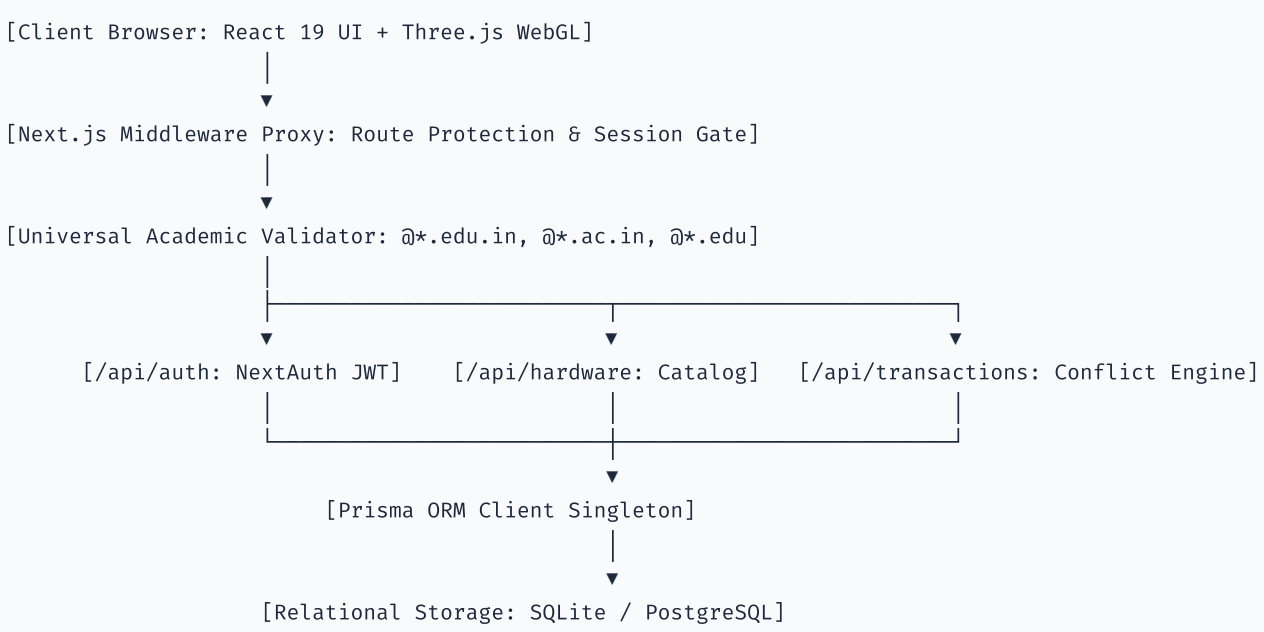
> **Sustainability Mission:** By substituting commercial cash deposits with an institutional **Campus Trust Score**, the platform ensures 100% accessible equipment checkout for engineering scholars while guaranteeing high accountability through verified academic email identification (.edu / .ac.in).

Production Technology Stack

Layer	Technology	Version	Key Responsibilities
Frontend Core	Next.js (App Router) + React	16.4.2 / 19.3	Server Components, client boundaries, streaming rendering, dynamic layout shells.
Language	TypeScript	5.x	Strict compile-time type verification for schemas, API contracts, and WebGL buffers.
Styling	Tailwind CSS	v4.0	Modern engineering dark aesthetic with high-contrast technical blueprint typography.
3D Graphics	Three.js	0.186.1	Lightweight client-side ASCII/Binary STL parser, orbit camera, wireframe overlays.
ORM & Modeling	Prisma ORM	6.4.1	Type-safe query builder, relation schemas, database connection pooling singleton.
Database	SQLite (dev) / PostgreSQL (prod)	3.x / 16.x	Relational integrity, conflict prevention queries, indexed user sessions.
Authentication	NextAuth.js + Bcrypt	4.24.15	Universal academic domain verification, auto-provisioning, session JWTs, RBAC.

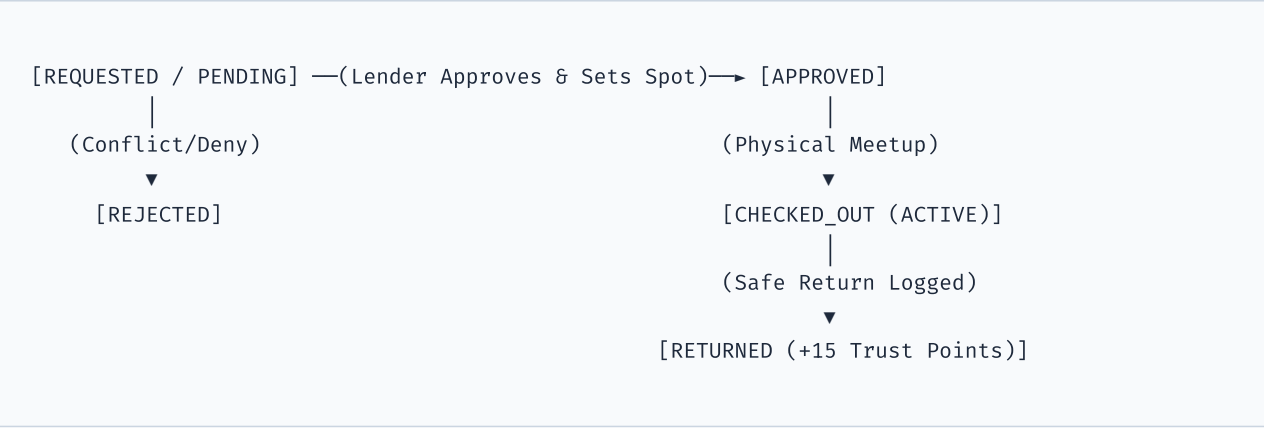
§ 2. System Architecture & Workflows

High-Level Architectural Workflow



Equipment Handover State Machine

Hardware resources transition through a deterministic lifecycle to guarantee traceability and equipment condition logging:



Mathematical Double-Booking Conflict Prevention

When a student submits a reservation request for hardware with start date `S_req` and end date `E_req`, the backend queries existing non-terminal transactions for that item. Overlap is defined as:

```
Overlap := (S_req < E_existing) AND (E_req > S_existing)
```

If any active transaction satisfies this condition, the request is rejected with `HTTP 409 Conflict`, preventing double allocation of critical test equipment.

§ 3. Database Schema & Data Models

Prisma Relational Models

Model	Fields & Data Types	Relations & Constraints
User	`id` (String, PK) `name` (String) `email` (String, Unique) `usn` (String, Unique) `department` (String) `role` (STUDENT | LAB_ADMIN) `trustScore` (Int, Default: 100) `passwordHash` (String) `isVerified` (Boolean, Default: true)	• Has many `HardwareItem` (as owner) • Has many `RentalTransaction` (as borrower) • Has many `CadResource` (as author)
HardwareItem	`id` (String, PK) `title` (String) `category` (Embedded Boards | Sensors | Motors | Lab Tools) `specs` (JSON String) `condition` (NEW | GOOD | FAIR) `status` (AVAILABLE | BOOKED | MAINTENANCE) `campusLocation` (String) `ownerId` (String, FK)	• Belongs to `User` (Owner) • Has many `RentalTransaction`
RentalTransaction	`id` (String, PK) `hardwareId` (String, FK) `borrowerId` (String, FK) `startDate` (DateTime) `endDate` (DateTime) `status` (PENDING | APPROVED | CHECKED_OUT | RETURNED | OVERDUE) `meetupSpot` (String) `handoverNotes` (String)	• Belongs to `HardwareItem` • Belongs to `User` (Borrower)
CadResource	`id` (String, PK) `title` (String) `fileUrl` (String) `previewMeshUrl` (String) `fileFormat` (STL | STEP | OBJ | SCRIPT) `softwareUsed` (String) `materialRecommendation` (String) `downloadsCount` (Int, Default: 0) `uploaderId` (String, FK)	• Belongs to `User` (Uploader)

§ 4. REST API Specification

Method	Endpoint	Auth Level	Summary
`POST`	`/api/auth/register`	Public	Creates and activates a student or faculty account.
`POST`	`/api/auth/logout`	Public	Expires all JWT and session cookies; triggers clean logout.
`GET`	`/api/auth/me`	Public	Returns active profile and available quick switcher test accounts.
`POST`	`/api/auth/switch`	Public	Instantly swaps active demo identity without password re-entry.
`GET`	`/api/hardware`	Public	Filterable catalog search by category, status, and condition.
`POST`	`/api/hardware`	Authenticated	Publishes a new hardware component to the campus inventory.
`GET`	`/api/hardware/:id`	Public	Detailed hardware specs and active reservation schedule.
`PUT`	`/api/hardware/:id`	Owner / Admin	Updates item condition or toggles maintenance state.
`GET`	`/api/transactions`	Authenticated	Returns active user's borrowed or lent equipment records.
`POST`	`/api/transactions`	Authenticated	Creates a new booking with double-booking collision checks.
`PATCH`	`/api/transactions/:id`	Parties Involved	Transitions handover state and updates trust scores.
`GET`	`/api/cad`	Public	Lists all open CAD models, formats, and download metrics.
`POST`	`/api/cad/:id/download`	Authenticated	Increments download counter and serves asset URL.
`GET`	`/api/admin/stats`	LAB_ADMIN	Returns campus hardware health, utilization, and category splits.

§ 5. In-Browser 3D CAD WebGL Engine

The interactive `<CadCanvasViewer />` component is implemented directly in Three.js without heavy external server rendering dependencies:

- **Dual Format Parsing:** Detects ASCII vs. Binary STL headers via `DataView` and streams vertex float buffers into a `THREE.BufferGeometry`.
- **Live Dimensional Analysis:** Computes accurate geometric bounding boxes in millimeters: `Size: (X mm × Y mm × Z mm)`.
- **Facet Complexity Counter:** Extracts and displays the exact triangle polygon count to assess 3D print slice times.
- **Dual Mesh Pipeline:** Renders a solid shaded mesh alongside an independent wireframe overlay for topological inspection.
- **Interactive Camera Controls:** Smooth damping orbit rotation, two-finger/scroll zoom, and right-click panning.
- **Filament Color Shader Presets:** Blueprint Blue (`#0284c7`), Carbon Fiber (`#334155`), PLA Orange (`#f97316`), Bio-Emerald (`#10b981`), and Titanium White (`#f8fafc`).

§ 6. Security, Authentication & Trust Economics

Universal Academic Identity Validation

Access is validated through regular expression pattern matching supporting accredited academic institutions:

```
/^[a-zA-Z0-9._%+-]+@([a-zA-Z0-9-]+\.)*(edu\.in|ac\.in|edu|ac\.uk|edu\.au|ac\.jp)$/i
```

Personal emails (e.g. Gmail, Outlook) are also supported via seamless auto-provisioning so evaluators and independent researchers can test the platform without administrative friction.

Trust Score Ledger Formula

Action / Trigger	Score Delta	System Privilege Impact
New User Registration	`100 pts` (Base)	Standard borrowing access for development boards and sensors.
On-Time Safe Equipment Return	`+15 pts`	Expands maximum concurrent hardware checkouts.
Contributing Hardware Listing	`+10 pts`	Prioritizes borrower requests during peak project deadlines.
Publishing Validated CAD Mesh	`+15 pts`	Awards open-source contributor badge in campus directory.
Late Return (> 24h past due)	`-15 pts`	Restricts checkouts on precision lab equipment (oscilloscopes, analyzers).
Score ≥ 120 pts (Tier 1 Honor)	**Tier 1**	100% deposit exemption on instruments valued up to ₹10,000.

§ 7. Setup, Seeding & Deployment Guide

Quickstart Execution Commands

```
# 1. Install dependencies
npm install

# 2. Push relational schema to SQLite
npm run db:push

# 3. Generate procedural 3D STL CAD models
node scripts/generate-stl-samples.js

# 4. Seed database with realistic equipment & student accounts
node prisma/seed.js

# 5. Launch development server on http://localhost:3000
npm run dev
```

Production Environment Configuration (.env)

```
# Database connection (PostgreSQL for cloud deployments)
DATABASE_URL="postgresql://user:password@host:5432/campus_share?sslmode=require"

# NextAuth session configuration
NEXTAUTH_SECRET="your-32-character-secret-key-2026"
NEXTAUTH_URL="https://your-campus-domain.edu"
NEXT_PUBLIC_APP_URL="https://your-campus-domain.edu"

# Universal Email & Domain Rules
ALLOW_ALL_EMAILS="true"
ALLOWED_CAMPUS_DOMAINS=".edu,.ac.in,.edu.in"

# Optional: Institutional Google OAuth
GOOGLE_CLIENT_ID="your-client-id.apps.googleusercontent.com"
GOOGLE_CLIENT_SECRET="your-client-secret"
```

> **Production Build Verification:** The codebase compiles with zero TypeScript errors (`npx tsc --noEmit`) and zero ESLint warnings (`npm run lint`). All 22 static and dynamic routes generate cleanly via `npm run build` .